

MLflow Intro Demo

Introduction

How to track, compare, version, and manage machine learning models using MLflow, including parameters, metrics, artifacts, and lifecycle stages.

Motivation

Suppose you have a company that works on spam detection task. You have two people who are working on building a spam detection classifier, Alice and Bob. Alice is working with traditional ML algorithms such as Random Forest etc and Bob is working with Neural Networks algorithms. Both of them are working on the same problem but with parallel tracks.

Both of them are working with the same dataset. But Each dataset has many versions. different models use maybe different processed dataset. Maybe after a while, a new feature comes from the higher-up, lets call it “emotional_score”. Now you have a new feature to your dataset. They name it dataset_2 that has the new feature. Then they retrain their models, run the notebooks, log their numbers of recall, accuracy etc.

One day, the boss comes and wants to know the best model. Both of them now has lots of notebooks, too many bloated codes, scattered numbers. Sometimes they won't even know which best model was trained on which best version of the dataset. What would be the solution?

Well they can create an excel file where they input all the information,. model name, dataset version, precision, recall, accuracy altogether. But an excel file is not actionable. Suppose the CEO wants to directly choose a model from the excel file and send it to the production? There is no send to production button in an excel file! And also, excel file is not UI friendly!

Thats why MLFlow is powerful. It can address all the above issues seamlessly.

Prerequisites

- Python 3.10+
- Git installed
- Virtual environment recommended

1 Conceptual framing (say this before typing)

- Git versions **code**
- DVC versions **data**
- MLflow versions **models** + **experiments**

A **model version** is not just a file — it includes:

- Model artifact (weights)
- Hyperparameters
- Evaluation metrics
- Dataset reference
- Code snapshot

MLflow stores all of this together.

2 Setup: Environment and MLflow UI

2.1 Create project and virtual environment

1. Create project folder:

```
mkdir mlflow-intro-demo
cd mlflow-intro-demo
```

2. Create and activate virtual environment:

macOS/Linux

```
python -m venv .venv
source .venv/bin/activate
```

Windows PowerShell

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

3. Install dependencies:

```
pip install mlflow pandas scikit-learn
```

2.2 Start MLflow tracking server

1. In the project root, run:

```
mlflow ui
```

2. Open browser:

```
http://127.0.0.1:5000/
```

3. Explain:

- This UI shows experiments and model versions
- Everything logged locally is visible here

3 Part A: First MLflow experiment

3.1 Create a simple training script

Create `src/train.py`:

```
import mlflow
import mlflow.sklearn
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

mlflow.set_tracking_uri("http://127.0.0.1:5000")

# Load simple dataset
df = pd.read_csv("data.csv")
X = df[["age", "income"]]
y = df["churned"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
with mlflow.start_run():
    model = LogisticRegression(max_iter=200)
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    mlflow.log_param("model_type", "logistic_regression")
    mlflow.log_param("max_iter", 200)
    mlflow.log_metric("accuracy", acc)
    mlflow.sklearn.log_model(
        model,
        "model",
    )
    print("Accuracy:", acc)
```

3.2 Create dataset

Create `data.csv`:

```
age,income,churned
25,42000,0
40,78000,1
19,25000,0
55,90000,0
33,61000,1
```

45,72000,1
28,38000,0

3.3 Run training

```
python src/train.py
```

Then show in the MLflow UI:

- A new run appears
- Params, metrics, and artifacts logged

4 Part B: Understanding MLflow concepts

Explain clearly:

Run

One execution of training with a specific configuration.

Experiment

A collection of related runs.

Artifacts

Files produced by a run:

- Models
- Plots
- Reports

Metrics

Numeric performance indicators (accuracy, loss).

Params

Hyperparameters and configuration values.

5 Part C: Model versioning via Model Registry

5.1 Register the model

Modify `train.py`:

```
mlflow.sklearn.log_model(  
    model,  
    "model",  
    registered_model_name="CustomerChurnModel"  
)
```

Re-run:

```
python src/train.py
```

5.2 Inspect Model Registry

In MLflow UI:

- Click **Models**
- Observe:
 - Model name: CustomerChurnModel
 - Version: v1
 - Source run

Explain:

A model version is immutable — once created, it never changes.

6 Part D: Create multiple model versions

6.1 Change hyperparameter

Modify:

```
model = LogisticRegression(max_iter=500)  
mlflow.log_param("max_iter", 500)
```

Re-run training:

```
python src/train.py
```

6.2 Observe in UI

- CustomerChurnModel now has:
 - Version 1
 - Version 2
- Each version:
 - Linked to different run
 - Has different metrics

7 Best practices

- Always log:
 - Parameters
 - Metrics
 - Model artifact
- Never overwrite models — create new versions
- Tie model runs to:
 - Git commit hash
 - DVC dataset version (log as param)

8 Now do

Re-do everything here with the best practices: dvc, dvc pipelines, params.yaml and using this dataset. Make sure to take out the categorical columns and predict the purchased column using logistic regression, using the code snippet that I have on this file. By now, you should be able to do all the automation necessary to implement this pipeline with mlflow incorporated. Try out different parameter values of logistic regression and see which version gives the best accuracy. For more, check out logistic regression documentation [here](#).